

Lab 02

Instalasi Docker

Dalam kegiatan praktik ini, Anda akan melakukan instalasi dependensi paket yang dibutuhkan docker sebelum diinstalasi, menginstal Docker di semua node (Docker01 dan Docker02), dan menguji hasil instalasi yang telah dilakukan.

Langkah kerja:

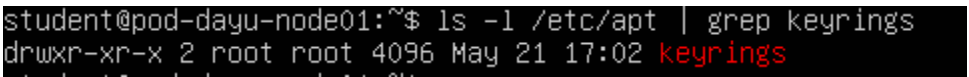
- A. [Persiapan](#)
- B. [Install Dependensi Docker](#)
- C. [Instalasi Docker](#)
- D. [Uji hasil instalasi](#)

A. Persiapan

1. Pastikan VM Docker01 dan Docker02 telah disiapkan sesuai Lab 01
2. Jalankan VM Docker01 dan Docker02 (boleh satu per satu)
3. Login ke masing-masing VM

B. Instal dan Konfigurasi Dependensi Docker

Jalankan di semua VM

1. Menghapus instalasi Docker versi lama (jika ada)
`sudo apt-get remove docker docker-engine docker.io containerd runc`
2. Memperbarui daftar paket
`sudo apt update`
3. Memperbarui semua paket yang telah terinstall
`sudo apt upgrade -y`
4. Menghapus paket yang tidak dibutuhkan
`sudo apt autoremove -y`
5. Menginstall dependensi untuk pengelolaan key dan repository docker
`sudo apt install -y ca-certificates curl gnupg lsb-release`
6. Membuat direktori untuk menyimpan key GPG docker
`sudo install -m 0755 -d /etc/apt/keyrings`
7. Verifikasi direktori yang telah dibuat
`ls -l /etc/apt | grep keyrings`

8. Mengunduh dan menyimpan GPG key Docker dalam format .gpg
`curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg`
9. Memberikan akses baca pada key tersebut
`sudo chmod a+r /etc/apt/keyrings/docker.gpg`

10. Menambahkan repository Docker resmi

```
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg]  
https://download.docker.com/linux/ubuntu $(lsb_release -cs)  
stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

11. Memuat ulang daftar paket agar mengenali repo baru

```
sudo apt update
```

C. Install Docker

Praktikkan di kedua VM (Docker01 dan Docker02)

1. Menginstall Docker engine dan plugin pendukung

```
sudo apt-get install -y docker-ce docker-ce-cli containerd.io  
docker-buildx-plugin docker-compose-plugin
```

2. Menampilkan versi docker yang telah terinstall

```
docker version
```

```
student@pod-dayu-node01:~/nusactl$ docker version  
Client: Docker Engine - Community  
Version: 26.1.3  
API version: 1.45  
Go version: go1.21.10  
Git commit: b72abbb  
Built: Thu May 16 08:33:29 2024  
OS/Arch: linux/amd64  
Context: default  
  
Server: Docker Engine - Community  
Engine:  
Version: 26.1.3  
API version: 1.45 (minimum version 1.24)  
Go version: go1.21.10  
Git commit: 8e96db1  
Built: Thu May 16 08:33:29 2024  
OS/Arch: linux/amd64  
Experimental: false  
containerd:  
Version: 1.6.32  
GitCommit: 8b3b7ca2e5ce38e8f31a34f35b2b68ceb8470d89  
runc:  
Version: 1.1.12  
GitCommit: v1.1.12-0-g51d5e94  
docker-init:  
Version: 0.19.0  
GitCommit: de40ad0
```

3. Menampilkan konfigurasi dan status Docker

```
docker info
```

```
student@pod-dayu-node01:~$ docker info
Client: Docker Engine - Community
Version:      26.1.3
Context:      default
Debug Mode:   false
Plugins:
  buildx: Docker Buildx (Docker Inc.)
    Version:  v0.14.0
    Path:      /usr/libexec/docker/cli-plugins/docker-buildx
  compose: Docker Compose (Docker Inc.)
    Version:  v2.27.0
    Path:      /usr/libexec/docker/cli-plugins/docker-compose

Server:
Containers: 1
  Running: 0
  Paused: 0
  Stopped: 1
Images: 3
Server Version: 26.1.3
Storage Driver: overlay2
  Backing Filesystem: extfs
  Supports d_type: true
  Using metacopy: false
  Native Overlay Diff: true
  userxattr: false
Logging Driver: json-file
Cgroup Driver: systemd
Cgroup Version: 2
Plugins:
  Volume: local
  Network: bridge host ipvlan macvlan null overlay
  Log: awslogs fluentd gcplogs gelf journald json-file local splunk syslog
Swarm: inactive
Runtimes: io.containerd.runc.v2 runc
Default Runtime: runc
Init Binary: docker-init
containerd version: 8b3b7ca2e5ce38e8f31a34f35b2b68ceb8470d89
runc version: v1.1.12-0-g51d5e94
init version: de40ad0
Security Options:
  apparmor
  seccomp
   Profile: builtin
  cgroupns
Kernel Version: 5.15.0-107-generic
Operating System: Ubuntu 22.04.4 LTS
OSType: linux
```

4. Menambahkan user aktif ke dalam grup Docker
Tujuannya agar saat menjalankan perintah docker tidak perlu menggunakan sudo
`sudo usermod -aG docker $USER`
5. Mengaktifkan grup Docker tanpa logout-login
`newgrp docker`

D. Uji Hasil Instalasi

1. Membuat dan menjalankan container
`docker run hello-world`
`docker run` : membuat/menjalankan container dari suatu image Docker. Jika image belum ada di mesin lokal, maka akan mengunduh dari registry yang ditentukan
`hello-world` : Nama dari image Docker yang ingin dijalankan.
2. Menampilkan image yang telah di download
`docker image ls`

```
student@pod-dayu-node01:~$ docker image ls
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
nginx	latest	1403e55ab369	19 months ago	142MB
httpd	latest	73c10eb9266e	19 months ago	145MB
registry.adinusa.id/btacade	latest	feb5d9fea6a5	2 years ago	13.3kB

3. Menampilkan semua container (aktif atau berstatus exit)
`docker container ls -a`

```
student@pod-dayu-node01:~$ docker container ls -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
a728988e5a0c	registry.adinusa.id/btacade	"/hello"	8 weeks ago	Exited (0) 8 weeks ago		laughing_poincar